

## 2. Requirement Analysis

### 2.1 Functional System Requirements

Functional parameters detail the explicit actions, parsing structures, and data handling workflows the application backend must perform to support network aggregation.

| System ID | Target Domain      | Functional Requirement Specification                                                                                          | Priority |
|-----------|--------------------|-------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-F-01  | Auth Architecture  | Provide cryptographically signed JSON Web Token (JWT) handling alongside secure OAuth2 layers for client verification.        | High     |
| REQ-F-02  | Profile Ingestion  | Ingest raw contact details via custom JSON payloads, text blocks, or structured vCard files, maps inputs to internal objects. | High     |
| REQ-F-03  | Semantic Mapping   | Extract keywords, primary business areas, and interaction histories using localized string vectorization pipelines.           | Medium   |
| REQ-F-04  | Proactive Warning  | Evaluate relationship states nightly and flag profiles dropping below target values.                                          | High     |
| REQ-F-05  | Drafting Assistant | Generate tone-adjusted communication templates using specific records from past interaction logs.                             | Medium   |

### 2.2 Non-Functional Architectural Constraints

Non-functional metrics establish performance baselines, security boundaries, and scalability guidelines for the application:

- **Performance & Speed:** Core dashboard query processing and data retrieval must complete within  $t \leq 250ms$  under normal user requests. NLP-driven template compilation must return text tokens within  $t \leq 1200ms$ .
- **Data Integrity & Security:** Protect personal contact details, interaction history, and communication strings at rest using AES-256 encryption rules, and secure data in transit via TLS 1.3.
- **Availability Profile:** Ensure the containerized service layer hits a target operational availability of 99.9% by leveraging automated cluster process management and isolated cache layers.

### 2.3 Engineering Environment Specifications

The selected technology components, data services, and development layers are structured to ensure absolute alignment across the application lifecycle:

| Architecture Layer  | Selected Framework / Component  | Engineering Rationale                                                                              |
|---------------------|---------------------------------|----------------------------------------------------------------------------------------------------|
| Client Interface    | React.js (v18) / Tailwind CSS   | Provides component-driven UI compilation with optimal state updates.                               |
| Backend Application | Python / FastAPI Server         | Delivers modern asynchronous routing with native input modeling via Pydantic.                      |
| Relational Database | PostgreSQL / pgvector extension | Maintains structured relational safety alongside hardware-accelerated text embedding matching.     |
| Caching Engine      | Redis Key-Value Cache Store     | Accelerates active user token verification and manages query rate limiters.                        |
| NLP Component       | HuggingFace Transformers API    | Manages local text matching and summary extraction tasks without depending on slow external calls. |